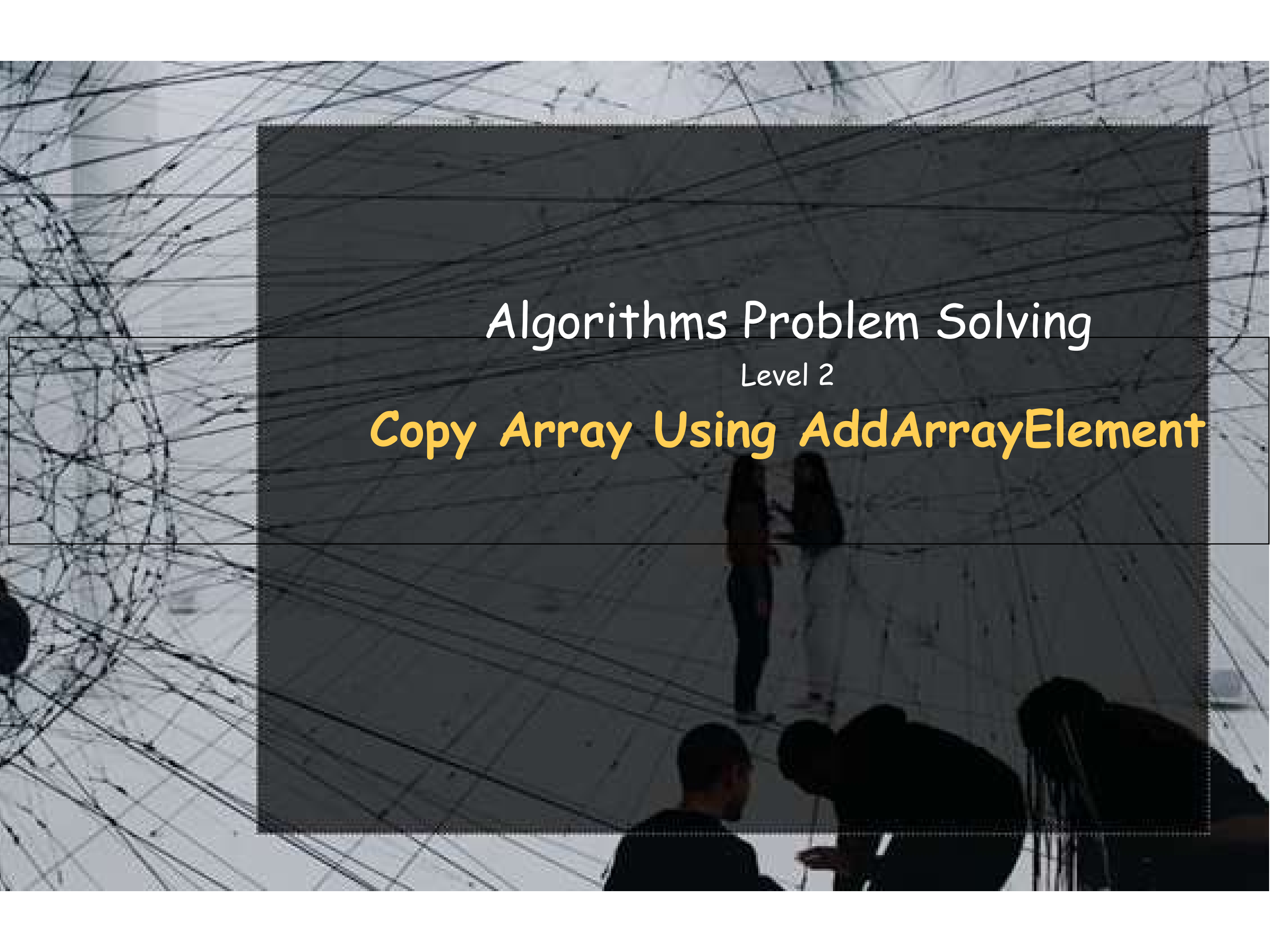


Algorithms Problem Solving

Level 2

Copy Array Using
AddArrayElement



Algorithms Problem Solving

Level 2

Copy Array Using AddArrayElement

Definition

- ◆ This program fills the first array with random numbers from `1` to `100`.
- ◆ It then copies all elements from the first array into a second array.
- ◆ The copying is done using `AddElementToArray()`.
- ◆ The second array starts with a length of `0` and grows as elements are added.
- ◆ Finally, both arrays are printed.

Explanation

- ◆ `arr1` contains the original random numbers.
- ◆ `arr2` starts empty:
 `int arr1Length = 0, arr1[100], arr2Length = 0, arr2[100];`
- ◆ `CopyArrayUsingAddElementToArray()` loops through every element in `arr1`:
 `for (int i = 0; i < arr1Length; i++)`
- ◆ For each element, it calls:
 `AddElementToArray(arr1[i], arr2, arr2Length);`
- ◆ `AddElementToArray()` increases `arr2Length` and stores the value at the next available index.
- ◆ The same process is repeated for every element in `arr1`.

☐ Steps in main()

- 1☐ Create `arr1` and `arr2`, both with a maximum size of `100`.
- 2☐ Set `arr1Length` and `arr2Length` to `0`.
- 3☐ Fill `arr1` with random numbers from `1` to `100`.
- 4☐ Print `arr1`.
- 5☐ Loop through all elements of `arr1`.
- 6☐ Send each element to `AddElementToArray()`.
- 7☐ Add the element to `arr2`.
- 8☐ Increase `arr2Length`.
- 9☐ Print `arr2`.

💡 Example

◆ `Array 1`:

42 68 35 1 70 25 79 59 63 65

◆ The program copies each element one by one:

42 → arr2[0]

68 → arr2[1]

35 → arr2[2]

1 → arr2[3]

70 → arr2[4]

...

65 → arr2[9]

◆ After the copying process, `arr2Length` becomes `10`.

➡ Result

Array 1 Elements: 42 68 35 1 70 25 79 59 63 65

Array 2 Elements after copy: 42 68 35 1 70 25 79 59 63 65

Note

◆ The important idea is that `CopyArrayUsingAddElementToArray()` does not directly assign:

```
arr2[i] = arr1[i];
```

◆ Instead, it uses the reusable `AddElementToArray()` function.

◆ `arr2Length` is passed by reference:

```
int& arr2Length
```

◆ This allows `AddElementToArray()` to update the actual length of `arr2`.

◆ `arr2Length` starts at `0` and increases every time an element is copied.

◆ This exercise combines the idea from the previous problem with array copying.

⊘ Warning

- ◆ The program should ensure that the entered number of elements does not exceed `100`.
- ◆ `arr2` must also have enough space for all copied elements.
- ◆ `ReadArrayElements()` is not used in this program.
- ◆ `cmath` and `corecrt\_math\_defines.h` are unnecessary.
- ◆ `srand()` and `time()` should ideally have their proper headers included explicitly.
- ◆ `FillArrayWithRandomNumber()` asks for the array length itself, so the initial value `arr1Length = 0` is only a starting value.



Comparison

`arr1`

- ✓ Contains the original random numbers.
- ✓ Its length is determined by the user.
- ✓ Acts as the source array.
- ✓ Its elements are not changed during copying.

`arr2`

- ✓ Starts empty with length `0`.
- ✓ Receives elements from `arr1`.
- ✓ Uses `AddElementToArray()` to add each element.
- ✓ Ends with the same elements and length as `arr1`.

✦ Summary

- ✓ Fill `arr1` with random numbers.
- ✓ Start `arr2Length` at `0`.
- ✓ Loop through all elements of `arr1`.
- ✓ Use `AddElementToArray()` to add each element to `arr2`.
- ✓ `AddElementToArray()` updates `arr2Length`.
- ✓ The final `arr2` contains the same elements as `arr1`.

Thank you 😊